BINARY REPRESENTATION

Module.1

CDA 3103C

Spring 2026

Mesut Ozdag, Ph.D.

Representations of Integers

- In the modern world, we use *decimal,* or *base* 10*, notation* to represent integers.

- We can represent numbers using any base b, where b is a positive integer greater than 1.

Base 10

- When we write 965, this can be translated as:
  - $9 \cdot 10^2 + 6 \cdot 10^1 + 5 \cdot 10^0$

Base 10

- When we write 965, this can be translated as:
  - $9 \cdot 10^2 + 6 \cdot 10^1 + 5 \cdot 10^0$

- $n = a_k b^k + a_{k-1} b^{k-1} + \ldots. + a_1 b + a_0$

Base b

- **Theorem:** Let b be a positive integer greater than 1. Then if n is a positive integer, it can be expressed uniquely in the form:

$$n = a_k b^k + a_{k-1} b^{k-1} + \ldots + a_1 b + a_0$$

  where k is a nonnegative integer and $a_0, a_1, \ldots a_k$ are nonnegative integers less than b.

- This representation of n is called the *base b expansion of n* and can be denoted by $(a_k a_{k-1} \ldots a_1 a_0)_b$.

- We usually omit the subscript 10 for base 10 expansions. ($(965)_{10}$)

Binary Expansions

- Computers represent integers and do arithmetic with binary (base 2) expansions of integers.
- In these expansions, the only digits used are $0 \text{ and } 1$.

Binary Expansions

- **Example**: What is the decimal expansion of the integer that has $(11011)_2$ as its binary expansion?
- **Solution**:

$(11011)_2$

$= 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$

$= 16 + 8 + 0 + 2 + 1$

$= 27.$

Binary Expansions

- **Example**: What is the decimal expansion of the integer that has $(1\ 0101\ 1111)_2$ as its binary expansion?
- **Solution**:

$$(1\ 0101\ 1111)_2$$
$$= 1{\cdot}2^8 + 0{\cdot}2^7 + 1{\cdot}2^6 + 0{\cdot}2^5 + 1{\cdot}2^4 + 1{\cdot}2^3 + 1{\cdot}2^2 + 1{\cdot}2^1 + 1{\cdot}2^0$$
$$= 256 + 0 + 64 + 0 + 16 + 8 + 4 + 2 + 1$$
$$=351.$$

Base Conversion

To construct the base b expansion of an integer n:

- Divide n by b to obtain a quotient and remainder.

  $n = bq_0 + a_0 \quad 0 \leq a_0 < b$

Base Conversion

To construct the base b expansion of an integer n:

- Divide n by b to obtain a quotient and remainder.

  $n = bq_0 + a_0 \quad 0 \leq a_0 < b$

- The remainder, a_0, is the rightmost digit in the base b expansion of n.

Base Conversion

To construct the base *b* expansion of an integer *n*:

- Divide *n* by *b* to obtain a quotient and remainder.

  $n = bq_0 + a_0 \quad 0 \leq a_0 < b$

- The remainder, a_0, is the rightmost digit in the base *b* expansion of *n*.
- Next, divide q_0 by *b*.

  $q_0 = bq_1 + a_1 \quad 0 \leq a_1 < b$

Base Conversion

To construct the base b expansion of an integer n:

- Divide n by b to obtain a quotient and remainder.

  $n = bq_0 + a_0 \quad 0 \leq a_0 < b$
- The remainder, a_0, is the rightmost digit in the base b expansion of n.
- Next, divide q_0 by b.

  $q_0 = bq_1 + a_1 \quad 0 \leq a_1 < b$
- The remainder, a_1, is the second digit from the right in the base b expansion of n.

Base Conversion

To construct the base *b* expansion of an integer *n*:

- Divide *n* by *b* to obtain a quotient and remainder.

  $n = bq_0 + a_0 \quad 0 \leq a_0 < b$

- The remainder, a_0, is the rightmost digit in the base *b* expansion of *n*.
- Next, divide q_0 by *b*.

  $q_0 = bq_1 + a_1 \quad 0 \leq a_1 < b$

- The remainder, a_1, is the second digit from the right in the base *b* expansion of *n*.
- Continue by successively dividing the quotients by *b*, obtaining the additional base *b* digits as the remainder. The process terminates when the quotient is 0.

Base Conversion

- **Example**: Find the binary expansion of $(19)_{10}$
- **Solution**: Successively dividing by 2 gives:
  - 19 = 2 * 9 + 1
  - 9 = 2 * 4 + 1
  - 4 = 2 * 2 + 0
  - 2 = 2 * 1 + 0
  - 1 = 2 * 0 + 1

Base Conversion

- **Example**: Find the binary expansion of $(19)_{10}$
- **Solution**: Successively dividing by 2 gives:
  - 19 = 2 * 9 + 1
  - 9 = 2 * 4 + 1
  - 4 = 2 * 2 + 0
  - 2 = 2 * 1 + 0
  - 1 = 2 * 0 + 1
- The remainders are the digits: read from bottom to top to become the binary number from left to right.
- $(10011)_2$.

Base Conversion

- **Example**: Find the binary expansion of $(12345)_{10}$
- **Solution**: Successively dividing by 2 gives:
  - 12345 = 2 * 6172 + 1
  - 6172 = 2 * 3086 + 0
  - 3086 = 2 * 1543 + 0
  - 1543 = 2 * 771 + 1
  - 771 = 2 * 385 + 1
  - 385 = 2 * 192 + 1
  - 192 = 2 * 96 + 0
  - 96 = 2 * 48 + 0
  - 48 = 2 * 24 + 0
  - 24 = 2 * 12 + 0
  - 12 = 2 * 6 + 0
  - 6 = 2 * 3 + 0
  - 3 = 2 * 1 + 1
  - 1 = 2 * 0 + 1
- $(12345)_{10} = (11\ 0000\ 0011\ 1001)_2$

List of Binary Numbers

Decimal	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

- These are 4-bit *unsigned* binary numbers.

Binary Addition

- Binary number can be added the same way decimal numbers are added:

Binary Addition

- Binary number can be added the same way decimal numbers are added:

- 3 + 2 = 5

	0	0	1	1
+	0	0	1	0

Binary Addition

- Binary number can be added the same way decimal numbers are added:

- 3 + 2 = 5

	0	0	1	1
+	0	0	1	0
				1

Binary Addition

- Binary number can be added the same way decimal numbers are added:

- 3 + 2 = 5

		1		
	0	0	1	1
+	0	0	1	0
			0	1

Binary Addition

- Binary number can be added the same way decimal numbers are added:

- 3 + 2 = 5

		1		
	0	0	1	1
+	0	0	1	0
		1	0	1

Binary Addition

- Binary number can be added the same way decimal numbers are added:

- 3 + 2 = 5

	0	0	1	1
+	0	0	1	0
	0	1	0	1

Binary Addition

- Binary number can be added the same way decimal numbers are added:

- 3 + 3 = 6

	0	0	1	1
+	0	0	1	1

Binary Addition

- Binary number can be added the same way decimal numbers are added:

- 3 + 3 = 6

$$
\begin{array}{ccccc}
 & & & 1 & \\
 & 0 & 0 & 1 & 1 \\
+ & 0 & 0 & 1 & 1 \\
\hline
 & & & & 0
\end{array}
$$

Binary Addition

- Binary number can be added the same way decimal numbers are added:

- 3 + 3 = 6

		1	1	
	0	0	1	1
+	0	0	1	1
			1	0

Binary Addition

- Binary number can be added the same way decimal numbers are added:

- 3 + 3 = 6

		1	1	
	0	0	1	1
+	0	0	1	1
		1	1	0

Binary Addition

- Binary number can be added the same way decimal numbers are added:
- 3 + 3 = 6

	0	0	1	1
+	0	0	1	1
	0	1	1	0

Binary Subtraction

- How do we subtract numbers?
- $7 - 6 = 7 + (-6) = 1$
- We need a way to represent negative numbers.

Signed Binary Numbers

- To represent a negative number, we use a sign bit.
- The sign bit is the most significant bit (MSB):
  - 1 represents a negative number.
  - 0 represents a positive number.

Representations for negative numbers:

- **2’s Complement** is the standard representation for negative numbers.

- Alternative representations:
  - **Sign-Magnitude**: sign bit + absolute value
    - Positive 7 0111
    - Negative 7 1111
  - **1’s Complement**: bitwise inverse
    - Positive 7 0111
    - Negative 7 1000

List of Binary Numbers

Decimal	Binary	Decimal	Binary
0	0000	-1	1111
1	0001	-2	1110
2	0010	-3	1101
3	0011	-4	1100
4	0100	-5	1011
5	0101	-6	1010
6	0110	-7	1001
7	0111	-8	1000

2's Complement Representation

- To represent a **negative** value: do a bitwise inverse, then add 1
- To determine -7:
  - 7: 0111
  - Bitwise inverse: 1000 *(1's complement)*
    Add 1: 1001

List of Binary Numbers

Decimal	Binary	Decimal	Binary
0	0000	-1	1111
1	0001	-2	1110
2	0010	-3	1101
3	0011	-4	1100
4	0100	-5	1011
5	0101	-6	1010
6	0110	-7	1001
7	0111	-8	1000

These are 4-bit signed numbers using the 2’s complement representation of negative numbers.
Biggest 4-bit number: 7
Smallest 4-bit number: -8

Zero Problem in 1's Complement

- 1's Complement Has TWO Zeros
  - +0 = 0000
  - -0 = 1111 ← this is a problem!
- 2's Complement Fixes This
  - +0 = 0000
  - -0 → invert 0000 → 1111
  - +1 → 0000 (back to single zero)

Summary:
Binary, Signed Integer Representations

B	Values represented		
$b_3 b_2 b_1 b_0$	Sign and magnitude	1's complement	2's complement
0 1 1 1	+ 7	+ 7	+ 7
0 1 1 0	+ 6	+ 6	+ 6
0 1 0 1	+ 5	+ 5	+ 5
0 1 0 0	+ 4	+ 4	+ 4
0 0 1 1	+ 3	+ 3	+ 3
0 0 1 0	+ 2	+ 2	+ 2
0 0 0 1	+ 1	+ 1	+ 1
0 0 0 0	+ 0	+ 0	+ 0
1 0 0 0	- 0	- 7	- 8
1 0 0 1	- 1	- 6	- 7
1 0 1 0	- 2	- 5	- 6
1 0 1 1	- 3	- 4	- 5
1 1 0 0	- 4	- 3	- 4
1 1 0 1	- 5	- 2	- 3
1 1 1 0	- 6	- 1	- 2
1 1 1 1	- 7	- 0	- 1

Binary Subtraction

- Binary numbers are subtracted by adding their 2’s complement equivalent.

- 7 – 6 = 7 + (-6) = 1

$$\begin{array}{ccccc} & 0 & 1 & 1 & 1 \\ + & 1 & 0 & 1 & 0 \\ \hline \end{array}$$

Binary Subtraction

- Binary numbers are subtracted by adding their 2's complement equivalent.

- 7 – 6 = 7 + (-6) = 1

6: 0110
Bitwise inverse: 1001
Add 1: 1010 (-6)

	0	1	1	1
+	1	0	1	0
				1

Binary Subtraction

- Binary numbers are subtracted by adding their 2's complement equivalent.

- 7 – 6 = 7 + (-6) = 1

		1		
	0	1	1	1
+	1	0	1	0
			0	1

Binary Subtraction

- Binary numbers are subtracted by adding their 2's complement equivalent.

- 7 – 6 = 7 + (-6) = 1

	1	1		
	0	1	1	1
+	1	0	1	0
		0	0	1

Binary Subtraction

- Binary numbers are subtracted by adding their 2's complement equivalent.

- 7 – 6 = 7 + (-6) = 1

1	1	1		
	0	1	1	1
+	1	0	1	0
	0	0	0	1

Binary Subtraction

- Binary numbers are subtracted by adding their 2's complement equivalent.

- 7 – 6 = 7 + (-6) = 1

1				
	0	1	1	1
+	1	0	1	0
	0	0	0	1

- The extra 1, called the carry-out, is ignored because the "Carry in" of the MSB is equal to the "Carry out.

Overflow

- Overflow occurs when the result of an arithmetic operation is too large or too small to represent.
  - In our 4-bit examples, that would occur if the result is less than -8 or greater than 7.

  - With five bits to represent magnitude:
    - As high as $+31_{10}$ (011111_2), or as low as -32_{10} (100000_2)

$-17_{10} = 101111_2$ $\quad$ $-19_{10} = 101101_2$

```
                 1 1111   <- - - Carry bits
(Showing sign bits)  101111
                   + 101101
                   - - - - - - -
                    1011100
                    |
            Discard extra bit
```

FINAL ANSWER: $011100_2 = +28_{10}$

Using the 7th bit:

$17_{10} + 19_{10}$ $\qquad$ $(-17_{10}) + (-19_{10})$

```
     1   11                  11  1111
    0010001                   1101111
  + 0010011                 + 1101101
  - - - - - - - -           - - - - - - - -
    0100100₂                 11011100₂
                             |
                       Discard extra bit
```

ANSWERS: $0100100_2 = +36_{10}$

$1011100_2 = -36_{10}$

Overflow

- Overflow occurs when adding whenever:
  - 2 positive numbers are added and the result is negative.
  - 2 negative numbers are added and the result is positive.

Operation	Operand A	Operand B	Result indicating overflow
$A + B$	≥ 0	≥ 0	< 0
$A + B$	< 0	< 0	≥ 0
$A - B$	≥ 0	< 0	< 0
$A - B$	< 0	≥ 0	≥ 0

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- 7 – 6 = 7 + (-6) = 1

Carry In

Carry Out				
1	1	1		
	0	1	1	1
+	1	0	1	0
	0	0	0	1

Most Significant Bit

Since the Carry in of the MSB is equal to the carry out, no overflow has occurred. The carry out is discarded and “0001” is the correct result.

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- 7 + 3 = 10

$$
\begin{array}{ccccc}
 & 0 & 1 & 1 & 1 \\
+ & 0 & 0 & 1 & 1 \\
\hline
\end{array}
$$

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- 7 + 3 = 10

			1	
	0	1	1	1
+	0	0	1	1
				0

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- 7 + 3 = 10

		1	1	
	0	1	1	1
+	0	0	1	1
			1	0

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- 7 + 3 = 10

	1	1	1	
	0	1	1	1
+	0	0	1	1
		0	1	0

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- 7 + 3 = 10

0	1	1	1	
	0	1	1	1
+	0	0	1	1
	1	0	1	0

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- 7 + 3 = 10

Carry Out

Carry In

0	1	1	1	
	0	1	1	1
+	0	0	1	1
	1	0	1	0

Most Significant Bit

Since the Carry in of the MSB is not equal to the carry out, an overflow has occurred.

The result calculated (-6) is clearly incorrect.

1010   (found)
↓ invert bits
0101
↓ add 1
0110.
0110 in decimal is 6.
So; the value is: ***−6***

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.
- -4 - 5 = -9

	1	1	0	0
+	1	0	1	1

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- -4 - 5 = -9

	1	1	0	0
+	1	0	1	1
				1

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- -4 - 5 = -9

	1	1	0	0
+	1	0	1	1
			1	1

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- -4 - 5 = -9

	1	1	0	0
+	1	0	1	1
		1	1	1

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- -4 - 5 = -9

1				
	1	1	0	0
+	1	0	1	1
	0	1	1	1

Overflow

- An easy way to spot overflow is to compare the "Carry in" of the MSB to the "Carry out". If they are different, then overflow has occurred.

- -4 - 5 = -9

Carry Out Carry In

	1	0			
		1	1	0	0
+		1	0	1	1
		0	1	1	1

Most Significant Bit

Since the Carry in of the MSB is not equal to the carry out, an overflow has occurred.

The result calculated (7) is clearly incorrect.

Overflow

- An easy way to spot overflow is to compare the "Carry in" of the MSB to the "Carry out". If they are different, then overflow has occurred.

- 3 + (-5) = -2

	0	0	1	1
+	1	0	1	1

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- 3 + (-5) = -2

			1	
	0	0	1	1
+	1	0	1	1
				0

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- 3 + (-5) = -2

		1	1	
	0	0	1	1
+	1	0	1	1
			1	0

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- 3 + (-5) = -2

$$
\begin{array}{ccccc}
 & 0 & 1 & 1 & \\
 & 0 & 0 & 1 & 1 \\
+ & 1 & 0 & 1 & 1 \\
\hline
 & & 1 & 1 & 0
\end{array}
$$

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- 3 + (-5) = -2

0	0	1	1	
	0	0	1	1
+	1	0	1	1
	1	1	1	0

Overflow

- An easy way to spot overflow is to compare the “Carry in” of the MSB to the “Carry out”. If they are different, then overflow has occurred.

- 3 + (-5) = -2

Carry Out → Carry In

Carry Out	0	0	1	1	
		0	0	1	1
	+	1	0	1	1
		1	1	1	0

Most Significant Bit

Since the Carry in of the MSB is equal to the carry out, no overflow has occurred. The extra zero is discarded and “1110” is the correct result.

MESUT OZDAG, PH.D.
MESUT.OZDAG@UCF.EDU